A Deep Neural Network from Scratch

Lecture 7: Neural networks, Part 7b

Khiem Nguyen

28 October 2026

James Watt School of Engineering, University of Glasgow

Table of contents

1 Objectives 1

2 Tools 2

3 The network 2
- 3.1 Activation functions . . . 2
- 3.2 Initialisation . . . 2
- 3.3 Forward pass . . . 3
- 3.4 Cost . . . 4
- 3.5 Backward pass . . . 4
- 3.6 Checking the gradients . . . 5

4 Vanishing gradients 5

5 Application: handwritten digits 7
- 5.1 Mini-batch gradient descent . . . 7

6 Challenges 9

7 Summary 9

1 Objectives

The previous tutorial built a network with one hidden layer. Here we build a network with **any number of layers**, again in NumPy only. You will

- write the forward and backward pass of a single layer, and chain them into a network with L layers;
- use a softmax output layer, so that the network can classify K classes;
- check all gradients with finite differences;
- observe vanishing gradients, and see how ReLU and He initialisation prevent them;
- train a deep network with mini-batch gradient descent on images of handwritten digits.

2 Tools

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

np.set_printoptions(precision=4, suppress=True)   # print arrays with 4 decimals,
 ↪  no e-notation
```

3 The network

A network with L layers computes, for $l = 1, \dots, L$ and starting from $\mathbf{A}^{[0]} = \mathbf{X}$ (one example per row),

$$\mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]}\big(\mathbf{W}^{[l]}\big)^\top + \mathbf{b}^{[l]}, \qquad \mathbf{A}^{[l]} = g^{[l]}\big(\mathbf{Z}^{[l]}\big), \qquad \mathbf{W}^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}, \quad \mathbf{b}^{[l]} \in \mathbb{R}^{n^{[l]}}.$$

We use ReLU (or sigmoid) in the hidden layers and **softmax** in the output layer, which turns the K scores of each example into K class probabilities (Lecture 3b). The architecture is given by the list of layer sizes $[n^{[0]}, n^{[1]}, \dots, n^{[L]}]$; for example, $[64, 128, 128, 10]$ is a network with 64 inputs, two hidden layers of 128 units and 10 outputs.

The lecture's picture is a chain of building blocks: each layer has a forward function, which also stores what the backward pass will need (the **cache**), and a backward function. We write exactly these blocks.

3.1 Activation functions

```
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

ACTIVATIONS = {                          # name: (g, g')
    'relu':    (lambda z: np.maximum(0, z), lambda z: (z > 0).astype(float)),
    'sigmoid': (sigmoid,                    lambda z: sigmoid(z) * (1 -
     ↪  sigmoid(z))),
}

def softmax(Z):
    """Row-wise softmax: every row of the result is a probability
     ↪  distribution."""
    Z = Z - Z.max(axis=1, keepdims=True)            # for numerical stability
    E = np.exp(Z)
    return E / E.sum(axis=1, keepdims=True)
```

3.2 Initialisation

All weights start with small random values, which breaks the symmetry between the units of a layer. How small matters for deep networks, as we will see. We provide two choices:

- `'he'`: **He initialisation** for ReLU layers, normal with standard deviation $\sqrt{2/n^{[l-1]}}$ and zero biases. The output layer uses standard deviation $\sqrt{1/n^{[l-1]}}$.

- `'default'`: PyTorch's default for `nn.Linear`, uniform on $[-1/\sqrt{n^{[l-1]}},\ 1/\sqrt{n^{[l-1]}}]$ for weights and biases.

```python
def initialize_parameters(layer_dims, method='he', seed=0):
    """Parameters W1, b1, ..., WL, bL for the layer sizes layer_dims = [n0, n1,
    ↪ ..., nL]."""
    rng = np.random.default_rng(seed)
    L = len(layer_dims) - 1
    params = {}
    for l in range(1, L + 1):
        n_in, n_out = layer_dims[l - 1], layer_dims[l]
        if method == 'he':
            std = np.sqrt(2 / n_in) if l < L else np.sqrt(1 / n_in)
            params[f'W{l}'] = rng.normal(0, std, (n_out, n_in))
            params[f'b{l}'] = np.zeros(n_out)
        else:
            k = 1 / np.sqrt(n_in)
            params[f'W{l}'] = rng.uniform(-k, k, (n_out, n_in))
            params[f'b{l}'] = rng.uniform(-k, k, n_out)
    return params

params = initialize_parameters([64, 128, 128, 10])
for name, value in params.items():
    print(f"{name}: shape {value.shape}")
```

```
W1: shape (128, 64)
b1: shape (128,)
W2: shape (128, 128)
b2: shape (128,)
W3: shape (10, 128)
b3: shape (10,)
```

3.3 Forward pass

The forward function of one layer returns its activations and a cache with its input $\mathbf{A}^{[l-1]}$ and its pre-activation $\mathbf{Z}^{[l]}$. The forward pass of the network applies the layers in order: the hidden activation in layers $1, \ldots, L-1$ and softmax in layer L.

```python
def layer_forward(A_prev, W, b, activation):
    """One layer: Z = A_prev W^T + b, A = g(Z). Returns A and the cache (A_prev,
    ↪ Z)."""
    Z = A_prev @ W.T + b
    A = softmax(Z) if activation == 'softmax' else ACTIVATIONS[activation][0](Z)
    return A, (A_prev, Z)

def forward(X, params, hidden='relu'):
    """Forward pass through all L layers. Returns the probabilities A^[L] (m, K)
    ↪ and the caches."""
    L = len(params) // 2
    A, caches = X, []
    for l in range(1, L + 1):
        A, cache = layer_forward(A, params[f'W{l}'], params[f'b{l}'], hidden if l
↪ < L else 'softmax')
        caches.append(cache)
```

```
    return A, caches

X_test = np.random.default_rng(0).normal(size=(5, 64))      # 5 examples, 64
 ↪  features
probs, caches = forward(X_test, params)
print(f"output shape {probs.shape}, row sums {probs.sum(axis=1)}")
```

```
output shape (5, 10), row sums [1. 1. 1. 1. 1.]
```

3.4 Cost

With the labels as one-hot rows $\mathbf{Y}$ (Lecture 3b), the cross-entropy is $J = -\frac{1}{m}\sum_i \sum_k Y_{ik} \log A^{[L]}_{ik}$. As before, we compute the logarithm of the softmax directly from $\mathbf{Z}^{[L]}$, which is numerically safe.

```
def compute_cost(caches, Y):
    """Cross-entropy from the output scores Z^[L] (stored in the last cache) and
     ↪  one-hot labels Y."""
    Z = caches[-1][1]
    Z = Z - Z.max(axis=1, keepdims=True)
    log_probs = Z - np.log(np.exp(Z).sum(axis=1, keepdims=True))
    return -np.mean(np.sum(Y * log_probs, axis=1))
```

3.5 Backward pass

For a softmax output with cross-entropy, the backward pass starts with $d\mathbf{Z}^{[L]} = \mathbf{A}^{[L]} - \mathbf{Y}$, exactly as for a sigmoid output in the previous tutorial. Then, for $l = L, \dots, 1$, the lecture's equations give

$$\frac{\partial J}{\partial \mathbf{W}^{[l]}} = \frac{1}{m}\left(d\mathbf{Z}^{[l]}\right)^{\top} \mathbf{A}^{[l-1]}, \qquad \frac{\partial J}{\partial \mathbf{b}^{[l]}} = \frac{1}{m}\sum_i \left(\text{row } i \text{ of } d\mathbf{Z}^{[l]}\right), \qquad d\mathbf{A}^{[l-1]} = d\mathbf{Z}^{[l]}\,\mathbf{W}^{[l]},$$

and, one layer further back, $d\mathbf{Z}^{[l-1]} = d\mathbf{A}^{[l-1]} \odot g'\left(\mathbf{Z}^{[l-1]}\right)$. The cache provides $\mathbf{A}^{[l-1]}$ and $\mathbf{Z}^{[l-1]}$, so nothing is recomputed.

```
def layer_backward(dZ, cache, W):
    """One layer backwards: from dZ to the gradients of W and b and to
     ↪  dA_prev."""
    A_prev, _ = cache
    m = A_prev.shape[0]
    dW = dZ.T @ A_prev / m
    db = dZ.sum(axis=0) / m
    dA_prev = dZ @ W
    return dA_prev, dW, db

def backward(probs, Y, params, caches, hidden='relu'):
    """Gradients of the cross-entropy with respect to all parameters."""
    L = len(caches)
    grads = {}
    dZ = probs - Y                                          # output layer:
 ↪  softmax + cross-entropy
    for l in range(L, 0, -1):
```

```
        dA_prev, grads[f'W{l}'], grads[f'b{l}'] = layer_backward(dZ, caches[l -
↪ 1], params[f'W{l}'])
        if l > 1:
            Z_prev = caches[l - 2][1]                          # Z^[l-1], from
↪ the cache of layer l-1
            dZ = dA_prev * ACTIVATIONS[hidden][1](Z_prev)
    return grads
```

3.6 Checking the gradients

We compare every gradient entry with a central finite difference, for a small network with three layers and random data:

```
def numerical_gradients(X, Y, params, hidden, eps=1e-6):
    """Central-difference approximation of all gradients, entry by entry."""
    grads = {}
    for name, value in params.items():
        grads[name] = np.zeros_like(value)
        for idx in np.ndindex(value.shape):
            original = value[idx]
            value[idx] = original + eps
            J_plus = compute_cost(forward(X, params, hidden)[1], Y)
            value[idx] = original - eps
            J_minus = compute_cost(forward(X, params, hidden)[1], Y)
            value[idx] = original
            grads[name][idx] = (J_plus - J_minus) / (2 * eps)
    return grads

rng = np.random.default_rng(1)
X_chk = rng.normal(size=(6, 4))
Y_chk = np.eye(3)[rng.integers(0, 3, size=6)]                # one-hot labels, K
↪ = 3
for hidden in ['relu', 'sigmoid']:
    p_chk = initialize_parameters([4, 5, 3, 3], seed=1)
    probs, caches = forward(X_chk, p_chk, hidden)
    grads = backward(probs, Y_chk, p_chk, caches, hidden)
    grads_fd = numerical_gradients(X_chk, Y_chk, p_chk, hidden)
    worst = max(np.max(np.abs(grads[k] - grads_fd[k])) for k in p_chk)
    print(f"{hidden:7s} hidden layers: largest difference over all gradients
    ↪ {worst:.1e}")
```

```
relu    hidden layers: largest difference over all gradients 1.5e-10
sigmoid hidden layers: largest difference over all gradients 2.8e-10
```

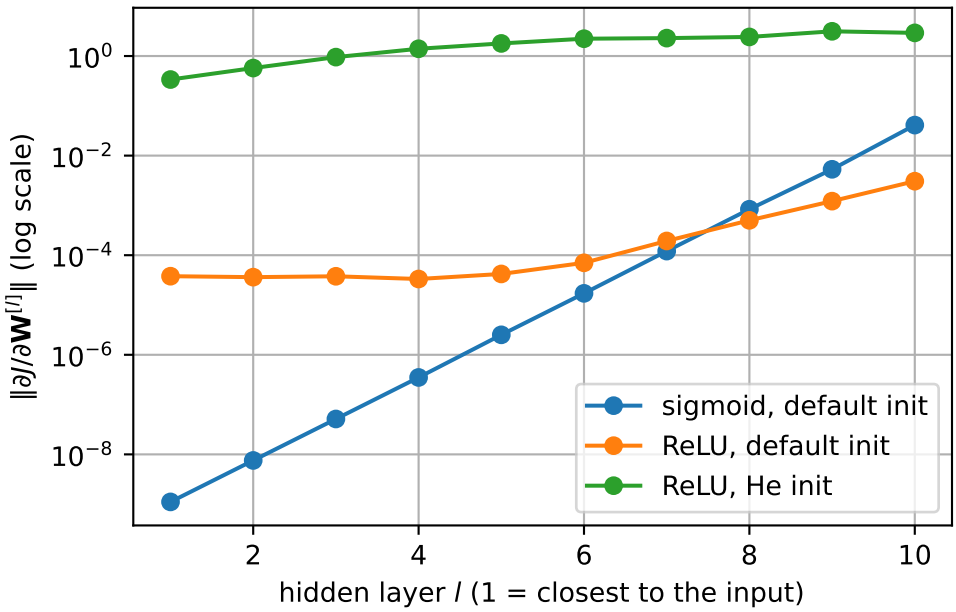
4 Vanishing gradients

Backpropagation multiplies by $g'(\mathbf{Z}^{[l]})$ and by $\mathbf{W}^{[l]}$ at every layer on its way back. If these factors are small, the gradient shrinks with every layer, and the first layers hardly learn. As in the lecture, we take a network with 10 hidden layers of 64 units, random data, and compare the size of $\partial J/\partial \mathbf{W}^{[l]}$ in every layer, at initialisation:

```
rng = np.random.default_rng(0)
X_rand = rng.normal(size=(256, 64))
Y_rand = np.eye(2)[rng.integers(0, 2, size=256)]
dims = [64] + [64] * 10 + [2]

for hidden, method, label in [('sigmoid', 'default', 'sigmoid, default init'),
                              ('relu', 'default', 'ReLU, default init'),
                              ('relu', 'he', 'ReLU, He init')]:
    p = initialize_parameters(dims, method)
    probs, caches = forward(X_rand, p, hidden)
    grads = backward(probs, Y_rand, p, caches, hidden)
    norms = [np.linalg.norm(grads[f'W{l}']) for l in range(1, 11)]
    plt.semilogy(range(1, 11), norms, 'o-', label=label)
    print(f"{label:22s}: layer 10 / layer 1 = {norms[-1] / norms[0]:.1e}")
plt.xlabel('hidden layer $l$ (1 = closest to the input)')
plt.ylabel('$\\|\\partial J / \\partial \\mathbf{W}^{[l]}\\|$ (log scale)')
plt.legend()
plt.grid(True)
plt.show()
```

```
sigmoid, default init : layer 10 / layer 1 = 3.7e+07
ReLU, default init    : layer 10 / layer 1 = 8.1e+01
ReLU, He init         : layer 10 / layer 1 = 8.6e+00
```



- **Sigmoid**: each step back multiplies by $\sigma'(z) \leq \frac{1}{4}$. The gradient of the first layer is millions of times smaller than that of the last hidden layer.
- **ReLU, default initialisation**: much better, but the gradient still shrinks by about two orders of magnitude.
- **ReLU, He initialisation**: the gradients of all layers have the same order of magnitude, so all layers learn at a similar speed.

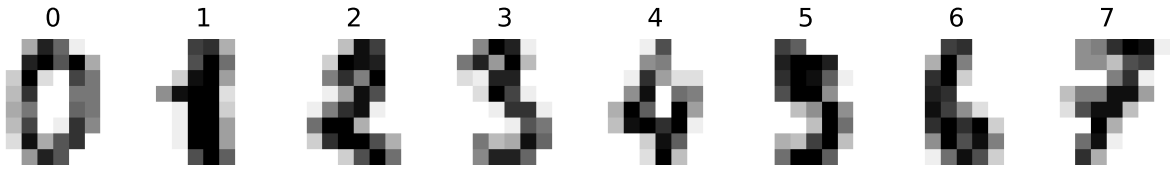
ReLU and a careful initialisation are what make deep networks trainable.

5 Application: handwritten digits

We now train a deep network on scikit-learn's handwritten digits: 1797 greyscale images of 8×8 pixels, each showing a digit from 0 to 9. As in the lecture's results slide, we keep 30% of the images for testing and use the architecture $64 \to 128 \to 128 \to 10$.

```
digits = load_digits()
fig, ax = plt.subplots(1, 8, figsize=(10, 1.6))
for a, image, label in zip(ax, digits.images, digits.target):
    a.imshow(image, cmap='gray_r')
    a.set_title(f'{label}')
    a.axis('off')
plt.show()

X = digits.data / 16.0                  # each image as a row of 64 pixels, scaled
 ↪  to [0, 1]
y = digits.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
 ↪  random_state=0, stratify=y)
Y_train = np.eye(10)[y_train]           # one-hot labels, shape (m, 10)
print(f"training set {X_train.shape}, test set {X_test.shape}")
```




```
training set (1257, 64), test set (540, 64)
```

Each image becomes a row of 64 pixel values, scaled from $0, \ldots, 16$ to $[0, 1]$.

5.1 Mini-batch gradient descent

Instead of using all training examples for every step, **mini-batch gradient descent** uses a random batch of 64 examples per step (Lecture 2a). One pass through all batches is an **epoch**. This is the same training scheme that PyTorch uses in the next tutorial.

```
def train(X, Y, layer_dims, hidden='relu', method='he', alpha=0.1, epochs=30,
 ↪  batch_size=64, seed=0):
    """Mini-batch gradient descent. Returns the parameters and the training cost
     ↪  after every epoch."""
    params = initialize_parameters(layer_dims, method, seed)
    rng = np.random.default_rng(seed)
    m = X.shape[0]
    J_history = []
    for epoch in range(epochs):
        order = rng.permutation(m)                          # shuffle the
 ↪  examples every epoch
        for start in range(0, m, batch_size):
            batch = order[start:start + batch_size]
            probs, caches = forward(X[batch], params, hidden)
            grads = backward(probs, Y[batch], params, caches, hidden)
```

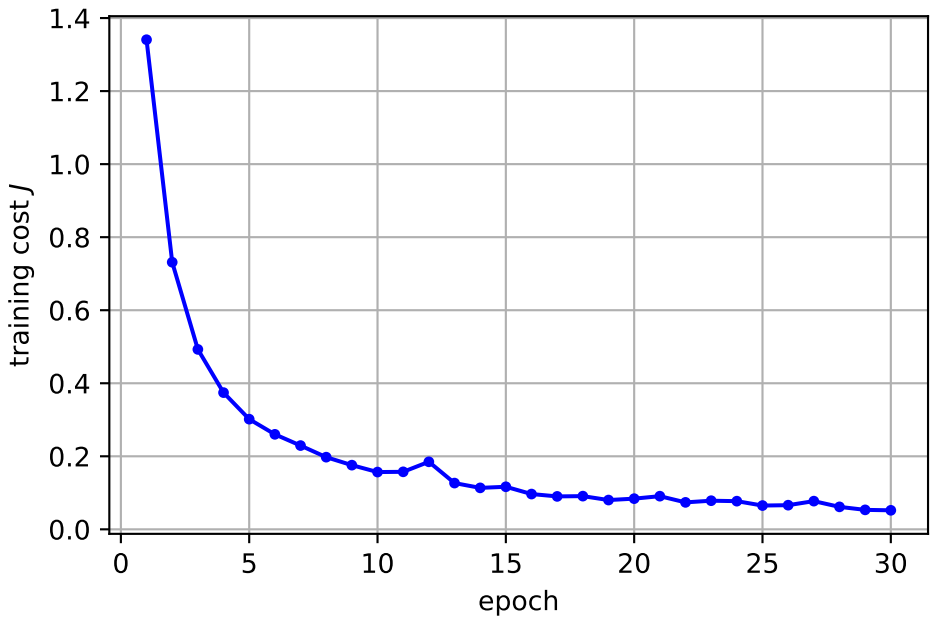
```
            for name in params:
                params[name] -= alpha * grads[name]
        J_history.append(compute_cost(forward(X, params, hidden)[1], Y))
    return params, J_history

def predict(X, params, hidden='relu'):
    """The class with the largest probability."""
    return forward(X, params, hidden)[0].argmax(axis=1)

params, J_hist = train(X_train, Y_train, [64, 128, 128, 10])
y_pred = predict(X_test, params)
print(f"test accuracy: {np.mean(y_pred == y_test):.3f}")

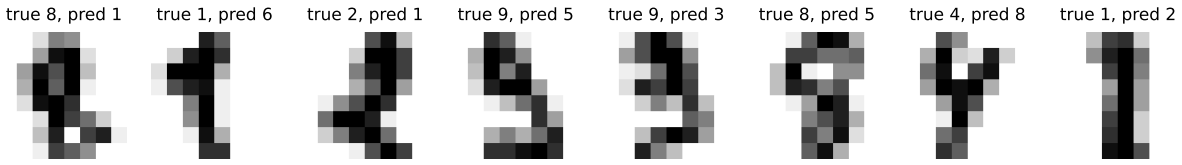
plt.plot(range(1, len(J_hist) + 1), J_hist, 'b-o', ms=3)
plt.xlabel('epoch')
plt.ylabel('training cost $J$')
plt.grid(True)
plt.show()
```

```
test accuracy: 0.969
```



Our NumPy network classifies about 97% of the test images correctly. Some of the mistakes are understandable, even for a human reader:

```
wrong = np.flatnonzero(y_pred != y_test)
fig, ax = plt.subplots(1, len(wrong[:8]), figsize=(10, 1.8))
for a, i in zip(np.atleast_1d(ax), wrong[:8]):
    a.imshow(X_test[i].reshape(8, 8), cmap='gray_r')
    a.set_title(f'true {y_test[i]}, pred {y_pred[i]}', fontsize=8)
    a.axis('off')
plt.show()
```



6 Challenges

1. Train with sigmoid hidden layers (`hidden='sigmoid', method='default'`). What happens to the training cost and the test accuracy, and why?
2. Train with `method='default'` and ReLU. How does the result compare with He initialisation?
3. Compare with scikit-learn's `LogisticRegression(max_iter=5000)` on the same split. What does the result tell you about these small 8×8 images? (The next tutorial uses larger images.)
4. Change the batch size to 1 (stochastic gradient descent) and to the whole training set (batch gradient descent), keeping 30 epochs. How do speed and accuracy change?

7 Summary

- A deep network is a chain of identical building blocks: a forward function that caches $\mathbf{A}^{[l-1]}$ and $\mathbf{Z}^{[l]}$, and a backward function that returns $\partial J/\partial \mathbf{W}^{[l]}$, $\partial J/\partial \mathbf{b}^{[l]}$ and $d\mathbf{A}^{[l-1]}$.
- With softmax and cross-entropy, as with sigmoid and binary cross-entropy, the backward pass starts from $d\mathbf{Z}^{[L]} = \mathbf{A}^{[L]} - \mathbf{Y}$.
- Through many sigmoid layers, gradients vanish. ReLU with He initialisation keeps them at the same size in every layer.
- Mini-batch gradient descent updates the parameters after every small batch; one pass through the data is an epoch.

Next: the same networks in PyTorch, where autograd computes the backward pass for us.